

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- die Grundprinzipien von
 - **Quicksort**,
 - **Heapsort** und
 - **Counting Sort**
- Die untere Schranke für das Sortierproblem bei Vergleichs-sortierv Verfahren

Sie können...

- die drei Sortierv Verfahren anwenden und nachvollziehen
- für eine gegebene Problemstellung ein geeignetes Sortierv Verfahren auswählen und begründen

Quicksort

Sortierv Verfahren

Quicksort

QUICKSORT(Z, l, r)

```
1: if  $l < r$  then  
2:    $q \leftarrow \text{PARTITION}(Z, l, r)$   
3:   QUICKSORT( $Z, l, q - 1$ )  
4:   QUICKSORT( $Z, q + 1, r$ )  
5: end if
```

Lösungsidee

Quicksort arbeitet nach dem Prinzip „Teile und Herrsche“: Er zerlegt die zu sortierende Datenmenge in zwei Teile und sortiert dann diese beiden Teile unabhängig voneinander.

Bei der Partitionierung muss allerdings sichergestellt werden, dass in der einen Teilmenge "kleinere" Elemente und in der anderen Teilmenge "größere" Elemente enthalten sind, um unabhängig sortieren zu können.

Dies geschieht durch (zufällige) Auswahl eines Pivot-Elements, das bei der Partitionierung als Vergleichswert für kleinere und größere Elemente herangezogen wird.

Sortiervverfahren

Quicksort – Partitionierung

Rollenspiel:

Stellen Sie sich "unsortiert" in einem Array auf. Eine geeignete Zahlenfolge ist:

4, 2, 7, 1, 6, 3, 8, 5

PARTITION(Z, l, r)

```
1:  $pivot \leftarrow Z[r]$ 
2:  $i \leftarrow l$ 
3:  $j \leftarrow r - 1$ 
4: while  $i < j$  do
5:   while  $Z[i] < pivot$  do
6:      $i \leftarrow i + 1$ 
7:   end while
8:   while  $j > l$  and  $Z[j] \geq pivot$  do
9:      $j \leftarrow j - 1$ 
10:  end while
11:  if  $i < j$  then
12:    SWAP( $Z[i], Z[j]$ )
13:     $i \leftarrow i + 1$ 
14:     $j \leftarrow j - 1$ 
15:  end if
16: end while
17: if  $i = j$  and  $Z[i] < pivot$  then
18:    $i \leftarrow i + 1$ 
19: end if
20: if  $Z[i] \neq pivot$  then
21:   SWAP( $Z[i], Z[r]$ )
22: end if
23: return  $i$ 
```

Sortiervverfahren

Quicksort – Partitionierung (Alternative)

Rollenspiel:

Stellen Sie sich "unsortiert" in einem Array auf. Eine geeignete Zahlenfolge ist:

4, 2, 7, 1, 6, 3, 8, 5

PARTITION2(Z, l, r)

```
1:  $pivot \leftarrow Z[r]$ 
2:  $i \leftarrow l - 1$ 
3: for  $j \leftarrow l$  to  $r - 1$  do
4:   if  $Z[j] \leq pivot$  then
5:      $i \leftarrow i + 1$ 
6:      $SWAP(Z[i], Z[j])$ 
7:   end if
8: end for
9:  $SWAP(Z[i + 1], Z[r])$ 
10: return  $i + 1$ 
```

Sortierv Verfahren

Quicksort - Laufzeit

QUICKSORT(Z, l, r)

```
1: if  $l < r$  then  
2:    $q \leftarrow \text{PARTITION}(Z, l, r)$   
3:   QUICKSORT( $Z, l, q - 1$ )  
4:   QUICKSORT( $Z, q + 1, r$ )  
5: end if
```

Effizienzüberlegung

Grundsätzlich: Es handelt sich um ein Divide-et-Impera-Verfahren, d.h.

- die Anzahl der Aufteilungen und
 - die Komplexität zur Ermittlung der Aufteilungen
- ist miteinander zu multiplizieren.

Die Anzahl der Aufteilungen hängt davon ab, wie gut das jeweils gewählte Pivot-Element die Menge partitioniert.

Sortiervverfahren

Quicksort - Laufzeit

Effizienzüberlegung Partitionierung

Auch wenn auf den ersten Blick verschachtelte Schleifen vorhanden sind, wird das gesamte Array nur einmal durchlaufen:

- i läuft von unten nach oben
- j läuft von oben nach unten

bis sich die beiden Indizes treffen. M.a.W.: die Additionen/Subtraktionen/Vergleiche werden in der Größenordnung n ausgeführt. Die anschließenden Verschiebungen fallen nicht ins Gewicht:

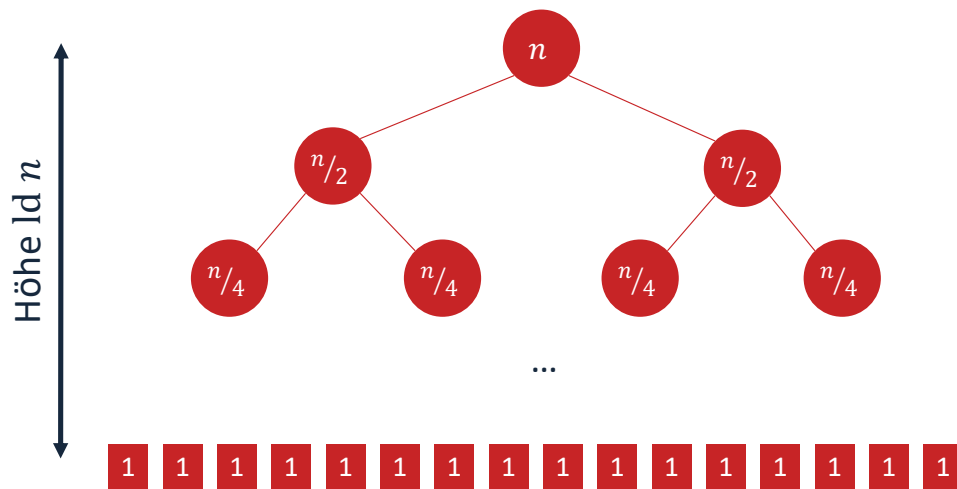
$$O(n)$$

PARTITION(Z, l, r)

```
1:  $pivot \leftarrow Z[r]$ 
2:  $i \leftarrow l$ 
3:  $j \leftarrow r - 1$ 
4: while  $i < j$  do
5:   while  $Z[i] < pivot$  do
6:      $i \leftarrow i + 1$ 
7:   end while
8:   while  $j > l$  and  $Z[j] \geq pivot$  do
9:      $j \leftarrow j - 1$ 
10:  end while
11:  if  $i < j$  then
12:    SWAP( $Z[i], Z[j]$ )
13:     $i \leftarrow i + 1$ 
14:     $j \leftarrow j - 1$ 
15:  end if
16: end while
17: if  $i = j$  and  $Z[i] < pivot$  then
18:    $i \leftarrow i + 1$ 
19: end if
20: if  $Z[i] \neq pivot$  then
21:   SWAP( $Z[i], Z[r]$ )
22: end if
23: return  $i$ 
```

Sortiervverfahren

Quicksort - Laufzeit



Effizienzüberlegung Rekursion

Im **Best Case** wird die Menge durch das Pivot-Element in der Mitte halbiert. Daraus ergibt sich ein Zerlegungsbaum der Höhe

$$\lg n$$

In jeder Zeile des Baums sind n Elemente für die nächste Partitionierung zu bearbeiten:

1. Zeile: n

2. Zeile: $n/2 + n/2 = n$

3. Zeile: $n/4 + n/4 + n/4 + n/4 = n$

...

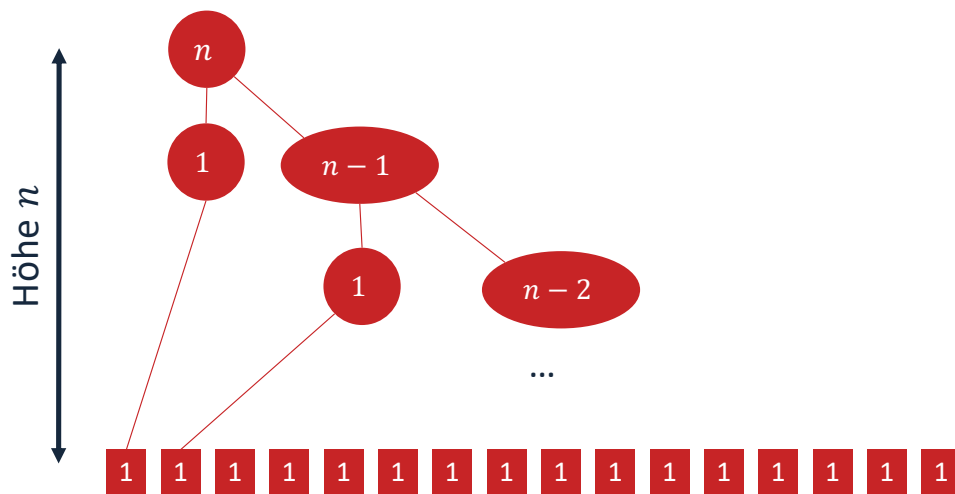
Letzte Zeile: $n \cdot 1 = n$

Da die Partitionierung mit $O(n)$ erfolgt, ergibt sich insgesamt

$$O(n \lg n)$$

Sortiervverfahren

Quicksort - Laufzeit



Effizienzüberlegung Rekursion

Im **Worst Case** teilt Partitionierung jeweils nur ein Element ab. Dadurch ergibt sich eine Höhe n für den Zerlegungsbaum.

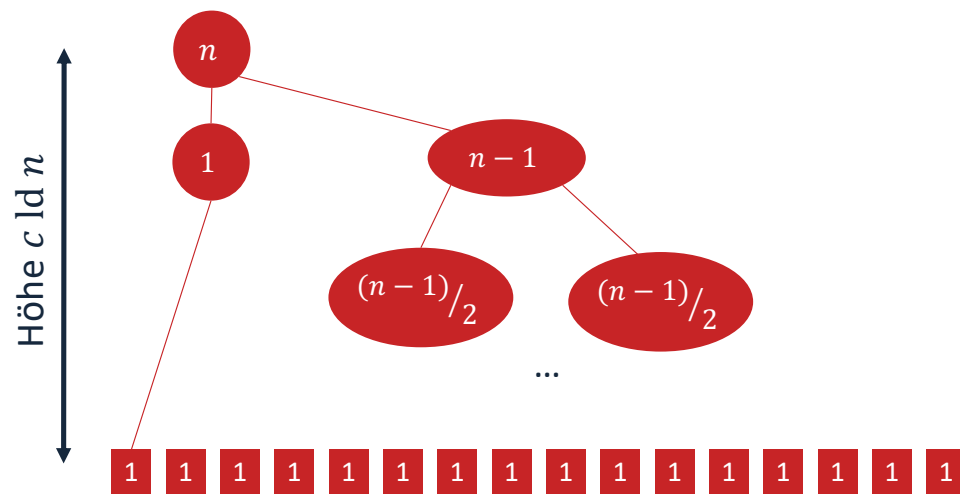
In jeder Zeile des Baums sind im Mittel $n/2$ Elemente für die nächste Partitionierung zu bearbeiten.

Da die Partitionierung mit $O(n)$ erfolgt, ergibt sich insgesamt

$$O(n^2)$$

Sortiervverfahren

Quicksort - Laufzeit



Effizienzüberlegung Rekursion

Im **Average Case** sind gute und schlechte Partitionierungen im Baum verteilt. Damit wird die Tiefe des Baums nicht n erreichen, sondern weiterhin mittels $c\ lg\ n$ mit $c > 1$ angegeben werden können.

Die genaue mathematische Analyse über alle Permutationen, die auftreten können, ergibt $c \approx 1,39$.

Es ergibt sich wiederum

$$O(n\ lg\ n)$$

Sortiervverfahren

Quicksort - Laufzeit

Sortiervverfahren	Worst Case	Average Case	Best Case
Quicksort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$

Hinweis:

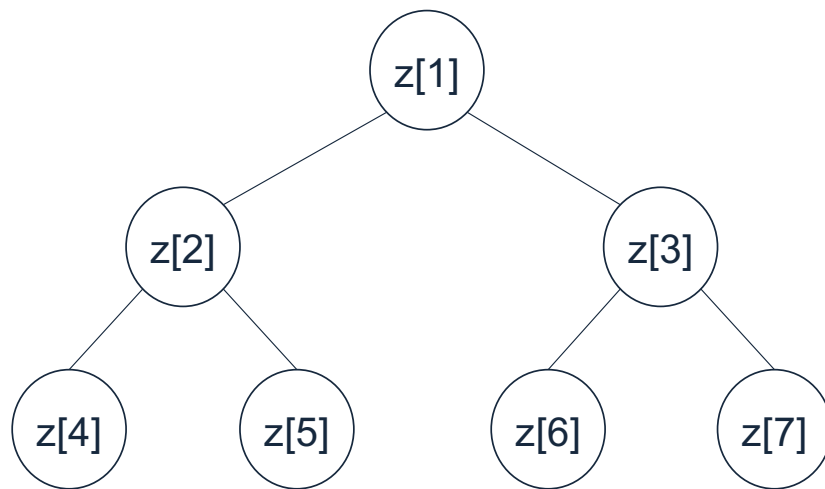
$$\log n = \frac{\lg n}{\lg 10}$$

M.a.W. zwischen den Logarithmen kann mit konstantem Faktor umgerechnet werden, d.h. für Landau-Symbole irrelevant.

Heapsort

Sortiervverfahren

Heapsort – Definition "Heap"



Unter einem **Heap** versteht man die Abbildung eines binären (d.h. jeder Knoten hat max. 2 Kinder) Baumes auf ein Array. Die Wurzel des Baumes wird im ersten Element des Arrays abgelegt. Die weiteren Knoten sind dann zeilenweise angeordnet.

Regeln für die Navigation in einem Heap

- Der Elternknoten eines Knoten i hat den Index $i/2$ (abgerundet)
- Der linke Kindknoten eines Knoten i hat den Index $2i$: $\text{LEFT}(i) \rightarrow 2i$
- Der rechte Kindknoten eines Knoten i hat den Index $2i+1$: $\text{RIGHT}(i) \rightarrow 2i+1$

Hinweis: Um die Berechnungen einfacher darstellen zu können, ist in diesem Kapitel der erste Arrayindex "1". Bei einer Implementierung ist dies entsprechend zu berücksichtigen.

Sortiervverfahren

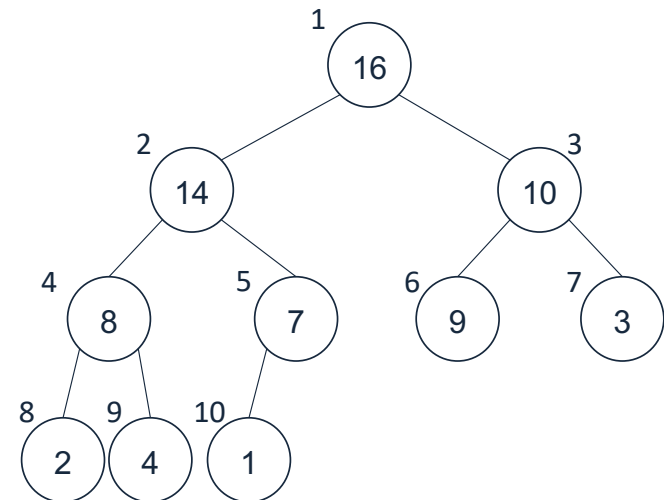
Heapsort – Definition "Max-Heap"

Ein Array erfüllt die **Max-Heap**-Eigenschaft, wenn jeder Elternknoten größer oder gleich seinen Kindknoten ist. Für jeden Knoten i gilt:

$$z[i] \geq z[2i] \text{ und } z[i] \geq z[2i + 1]$$

Beispielarray mit Max-Heap-Eigenschaft:

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Sortierverfahren

Heapsort

HEAPSORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$            ▷ Achtung:  $Z[1 \dots n]$ 
2: MAKE-MAX-HEAP( $Z$ )
3:  $\text{heapsize} \leftarrow n$ 
4: for  $i \leftarrow n$  downto 2 do
5:   SWAP( $Z[1], Z[i]$ )
6:    $\text{heapsize} \leftarrow \text{heapsize} - 1$ 
7:   MAX-HEAPIFY( $Z, 1, \text{heapsize}$ )
8: end for
```

Lösungsidee

Stelle im unsortierten Array durch Vertauschungen die Max-Heap-Eigenschaft her (MAKE-MAX-HEAP). Das erste Element ist dann das größte im Array.

Vertausche das erste Element (Maximum) mit dem letzten Element und stelle im verbliebenen Array (ohne das letzte Element) wieder die Max-Heap-Eigenschaft her (MAX-HEAPIFY).

Wiederhole den letzten Schritt, bis das Array sortiert ist.

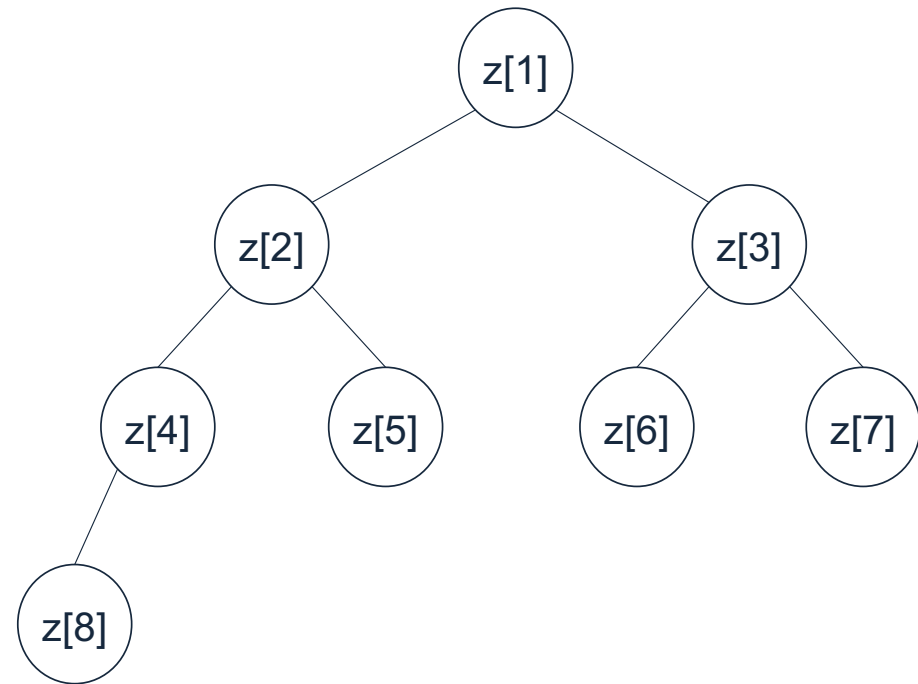
Sortiervverfahren

Heapsort – Herstellen der Max-Heap-Eigenschaft

Rollenspiel:

Stellen Sie sich "unsortiert" in einem Array auf.

Beginnend am Ende stellen wir die Max-Heap-Eigenschaft her, indem wir den Wert ggf. "absinken" lassen.

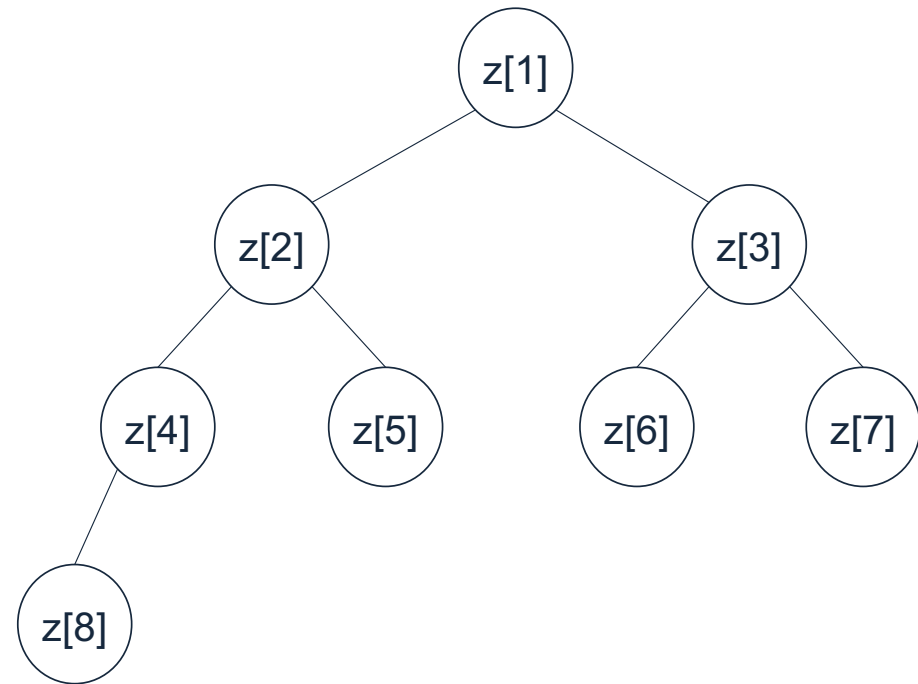


Sortiervverfahren

Heapsort – Tausch des größten Elements nach hinten

Rollenspiel:

Das erste Element geht ans Ende,
der zu sortierende Heap
verkleinert sich.



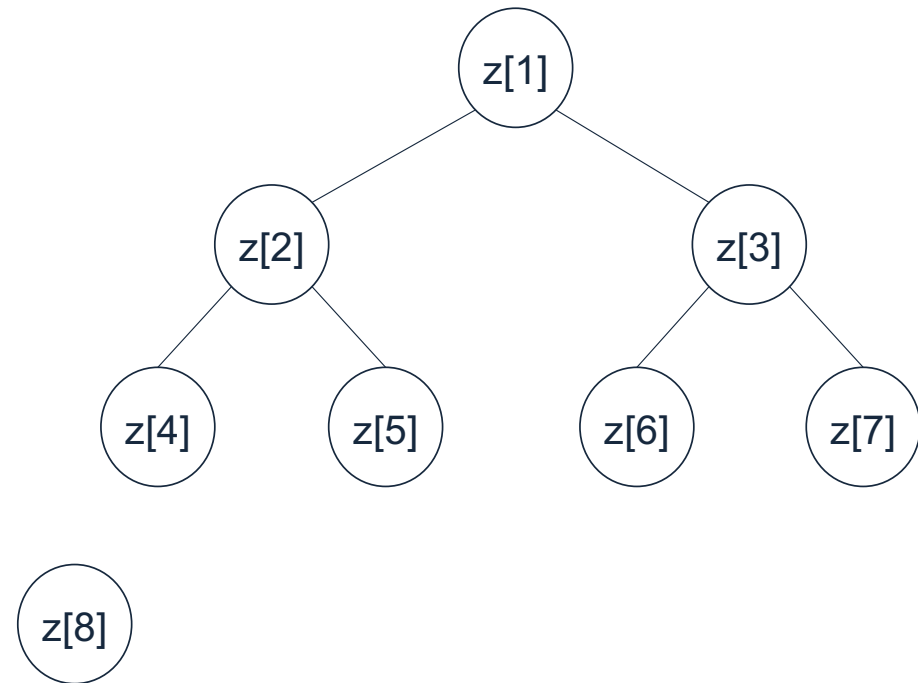
Sortiervverfahren

Heapsort – Wiederherstellen der Max-Heap-Eigenschaft

Rollenspiel:

Das an $z[1]$ eingefügte Element verstößt u.U. gegen die Max-Heap-Eigenschaft und muss absinken.

Danach wieder Tausch ans Ende.



Sortierv Verfahren

Heapsort – Wiederherstellen der Max-Heap-Eigenschaft

MAX-HEAPIFY($Z, i, \text{heapsize}$)

```
1:  $l \leftarrow \text{LEFT}(i)$            ▷ Linker Kindknoten
2:  $r \leftarrow \text{RIGHT}(i)$         ▷ Rechter Kindknoten
3:  $\text{max} \leftarrow i$ 
4: if  $l \leq \text{heapsize}$  and  $Z[l] > Z[i]$  then
5:    $\text{max} \leftarrow l$ 
6: end if
7: if  $r \leq \text{heapsize}$  and  $Z[r] > Z[\text{max}]$  then
8:    $\text{max} \leftarrow r$ 
9: end if
10: if  $\text{max} \neq i$  then
11:   SWAP( $Z[i], Z[\text{max}]$ )
12:   MAX-HEAPIFY( $Z, \text{max}, \text{heapsize}$ )
13: end if
```

Effizienzüberlegung

Die Tiefe des Heap ist $\ln n$, d.h. die Rekursion findet max. $\ln n$ mal statt.

Damit besitzt MAX-HEAPIFY eine Laufzeitkomplexität von $O(\log n)$

Hinweis: Die maximale Laufzeit wird erreicht, wenn das fehlerhafte Element ganz nach unten durchgereicht werden muss.

Sortiervverfahren

Heapsort – Initiales Herstellen der Max-Heap-Eigenschaft

MAKE-MAX-HEAP(Z)

```
1:  $n \leftarrow \text{length}(Z)$            ▷ Achtung:  $Z[1 \dots n]$ 
2: for  $i \leftarrow n/2$  downto 1 do
3:   MAX-HEAPIFY( $Z, i, n$ )
4: end for
```

Effizienzüberlegung

Mit zuvor vorgestellten MAX-HEAPIFY kann man ein Array $Z[1..n]$, das Werte in beliebiger Reihenfolge enthält, so umformen, dass es danach Max-Heap-Eigenschaft besitzt. Dies erreicht man mit dem nebenstehenden Algorithmus MAKE-MAX-HEAP.

Glücklicherweise ist der oberflächliche Betrachtung, dass $\frac{n}{2}$ mal MAX_HEAPIFY mit $O(\log n)$ falsch, weil viele der MAX_HEAPIFY-Aufrufe nur sehr geringe Höhen haben. Es ergibt sich für MAKE-MAX-HEAP eine Laufzeitkomplexität von

$$O(n)$$

Sortierv Verfahren

Heapsort

HEAPSORT(Z)

```
1:  $n \leftarrow \text{length}(Z)$            ▷ Achtung:  $Z[1 \dots n]$ 
2: MAKE-MAX-HEAP( $Z$ )
3:  $\text{heapsize} \leftarrow n$ 
4: for  $i \leftarrow n$  downto 2 do
5:   SWAP( $Z[1], Z[i]$ )
6:    $\text{heapsize} \leftarrow \text{heapsize} - 1$ 
7:   MAX-HEAPIFY( $Z, 1, \text{heapsize}$ )
8: end for
```

Effizienzbetrachtung

Da MAKE-MAX-HEAP eine Laufzeitkomplexität von $O(n)$ besitzt, dominiert die Schleife (Zeilen 4-8). Dort wird in der Größenordnung n mal MAX-HEAPIFY aufgerufen, das die Komplexität $O(\log n)$ besitzt. Somit ergibt sich insgesamt eine Laufzeitkomplexität von

$$O(n \log n)$$

Hinweis: Eine Begrenzung der Vertauschungen wie bei MAKE-MAX-HEAP kann hier nicht angegeben werden, weil bei jedem Durchlauf das zur Wurzel bewegte Element bis zu einem Blatt durchgereicht werden kann.

Sortiervverfahren

Heapsort - Laufzeit

Sortiervverfahren	Worst Case	Average Case	Best Case
Quicksort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Untere Schranke für das Sortierproblem

Sortiervverfahren

Untere Schranke für das Sortierproblem

- Wir haben Sortieralgorithmen identifiziert, die im Durchschnitt die Zeitkomplexität $O(n \log n)$ aufweisen.
- Ist dies bereits das Beste, was man erreichen kann?
- In diesem Fall wäre das Sortierproblem „ $\Omega(n \log n)$ -hart“
- Wir werden im Folgenden beweisen, dass allein mit Vergleichen im Rahmen einer Ordnungsrelation (z.B. $\leq$) kein besseres Verfahren existieren kann.

Sortierv Verfahren

Untere Schranke für das Sortierproblem

- **Annahme:** In einem Array der Länge n sind die Elemente $e_1 \dots e_n$ enthalten, die paarweise mit einer Ordnungsrelation $\leq$ verglichen werden können
- **Gesucht:** Die Permutation der Elemente, so dass paarweise zwischen allen Nachbarn die Ordnungsrelation gegeben ist.
- **Überlegung:** es gibt $n!$ Permutationen, von denen genau eine die Richtige ist.

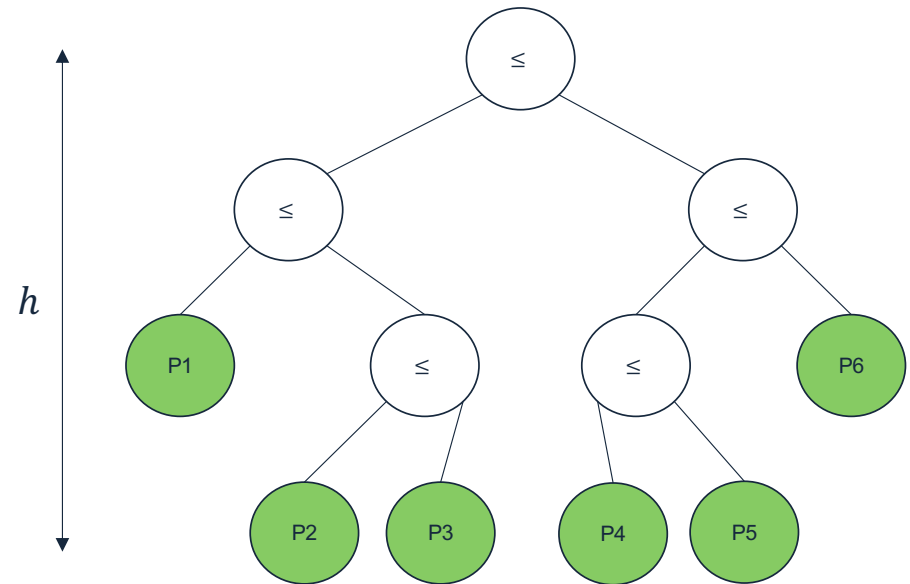
Zur Ermittlung der richtigen Permutation müssen Vergleiche ausgeführt und Entscheidungen getroffen werden.

→ **Entscheidungsbaum** mit den Permutationen als Blätter

Sortiervverfahren

Untere Schranke für das Sortierproblem

- **Entscheidungsbaum** für $n = 3$ mit $n! = 6$ Permutationen.
- Bei optimaler Anordnung der Entscheidungen entsteht ein binärer Baum mit den Permutationen P als Blätter.
- Im **Worst Case** muss zur Identifikation der richtigen Permutation die gesamte Höhe h des Baums durchlaufen werden.



Sortiervverfahren

Untere Schranke für das Sortierproblem

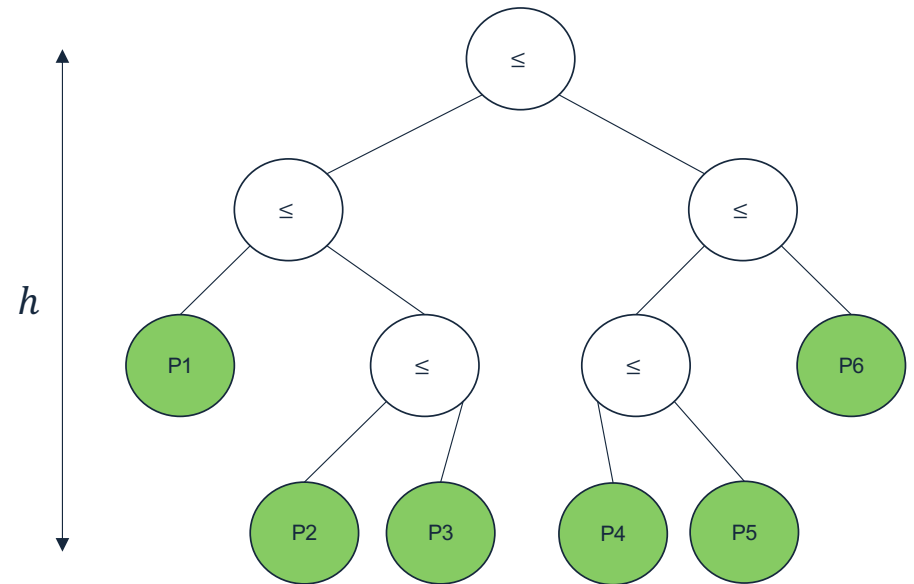
Wie hoch ist der Entscheidungsbaum?

Bei n Elementen ergeben sich $n!$ Permutationen. Der Entscheidungsbaum muss hoch genug sein, dass diese als Blätter geführt werden können.

Ein binärer Entscheidungsbaum der Höhe h hat maximal 2^h Blätter (wenn alle Zweige bis zum Ende durchgezogen werden).

Somit:

$$n! \leq 2^h$$
$$\lg n! \leq h$$



Sortiervverfahren

Untere Schranke für das Sortierproblem

Anzahl der Entscheidungen

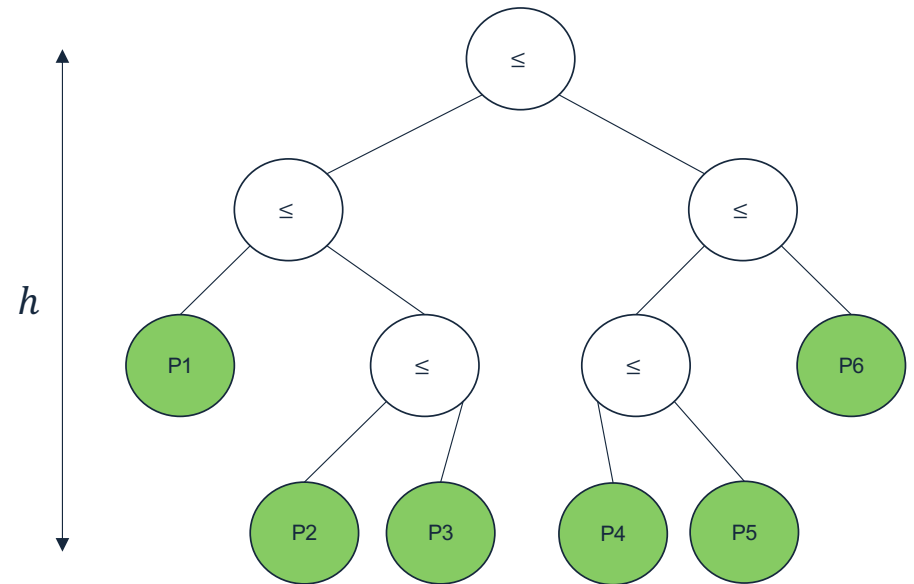
Somit:

$$n! \leq 2^h$$
$$\lg n! \leq h$$

Es müssen also in der Größenordnung $\lg n!$ Entscheidungen getroffen werden, um bei der richtigen Permutation anzukommen.

Ohne Beweis: $\lg n! \in \Theta(n \log n)$

⇒ **Sortierproblem ist „ $\Omega(n \log n)$ -hart“**



Counting Sort

Sortiervverfahren

Counting-Sort

Es wurde gezeigt, dass ein Sortiervverfahren, das ausschließlich auf Vergleiche abhebt, keine bessere Laufzeitkomplexität besitzen kann als

$$O(n \log n)$$

Wenn man aber weitere Vorbedingungen für die zu sortierenden Objekte gegeben sind, so kann auch ein besseres Laufzeitverhalten erreicht werden.

Vorbedingungen für den Counting-Sort

Um den Counting-Sort anwenden zu können, sind folgende Vorbedingungen notwendig:

- Sortiert werden positive ganze Zahlen bzw. das Sortierkriterium sind positive ganze Zahlen (d.h. es können noch weitere „Satelitendaten“ in einem struct o.Ä. existieren)
- Man kennt eine obere Schranke für die zu sortierenden Zahlen

Sortiervverfahren

Counting-Sort (Beispiel)

Die Funktionsweise des Counting-Sort soll an einem Beispiel aufgezeigt werden.

Die zu sortierenden Zahlen sind in einem Feld A enthalten; das Ergebnis soll anschließend in einem weiteren Feld B gleicher Größe bereitgestellt werden.
(d.h. es ist kein „in-place“-Algorithmus)

Die obere Schranke der zu sortierenden Zahlen sein k .

1. Schritt:

- Zu sortierendes Array A :

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

- Maximum der zu sortierenden Zahlen: $k = 5$

```
1:  $C \leftarrow \text{new Array}[0 \dots k]$   
2: for  $i \leftarrow 0$  to  $k$  do  
3:    $C[i] \leftarrow 0$   
4: end for
```

- Initiale Befüllung des Array C :

0	1	2	3	4	5
0	0	0	0	0	0

Sortiervverfahren

Counting-Sort (Beispiel)

2. Schritt:

Anschließend wird die Häufigkeit jedes Wertes gezählt und im Feld C vermerkt.

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

```
5: for  $j \leftarrow 1$  to  $length(A)$  do
6:    $C[A[j]] \leftarrow C[A[j]] + 1$ 
7: end for
```

0	1	2	3	4	5
2	0	2	3	0	1

Sortiervverfahren

Counting-Sort (Beispiel)

3. Schritt:

Nach einem nochmaligen Durchlaufen von C ist jetzt nicht mehr die Anzahl des jeweiligen Wertes, sondern die Anzahl der Werte kleiner oder gleich vermerkt.

```
8: for  $i \leftarrow 1$  to  $k$  do  
9:    $C[i] \leftarrow C[i] + C[i - 1]$   
10: end for
```

0	1	2	3	4	5
2	0	2	3	0	1

0	1	2	3	4	5
2	2	4	7	7	8

Sortiervverfahren

Counting-Sort (Beispiel)

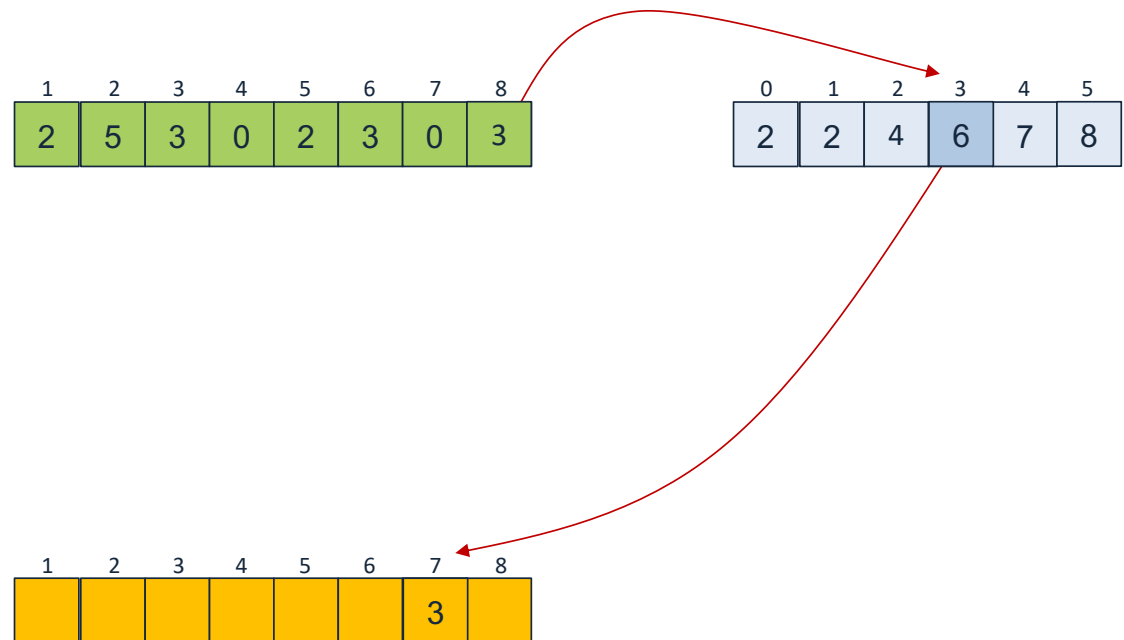
4. Schritt:

Zuletzt wird das Ergebnis-Feld B befüllt.

```

11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for

```



Sortiervverfahren

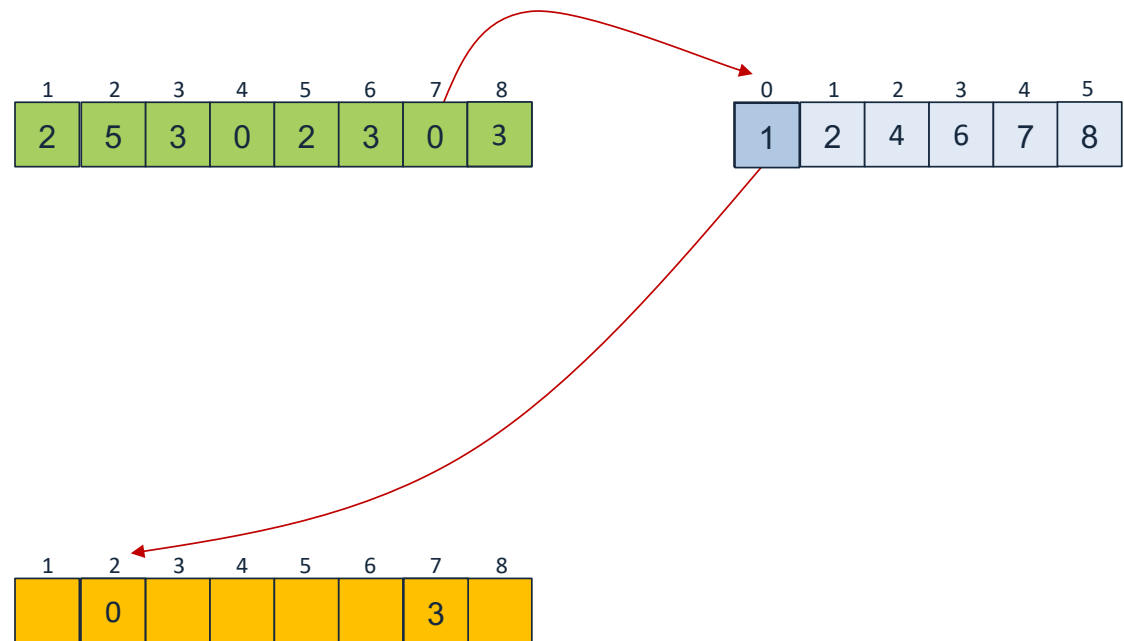
Counting-Sort (Beispiel)

4. Schritt:

Zuletzt wird das Ergebnis-Feld B befüllt.

```

11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for
    
```



Sortiervverfahren

Counting-Sort (Beispiel)

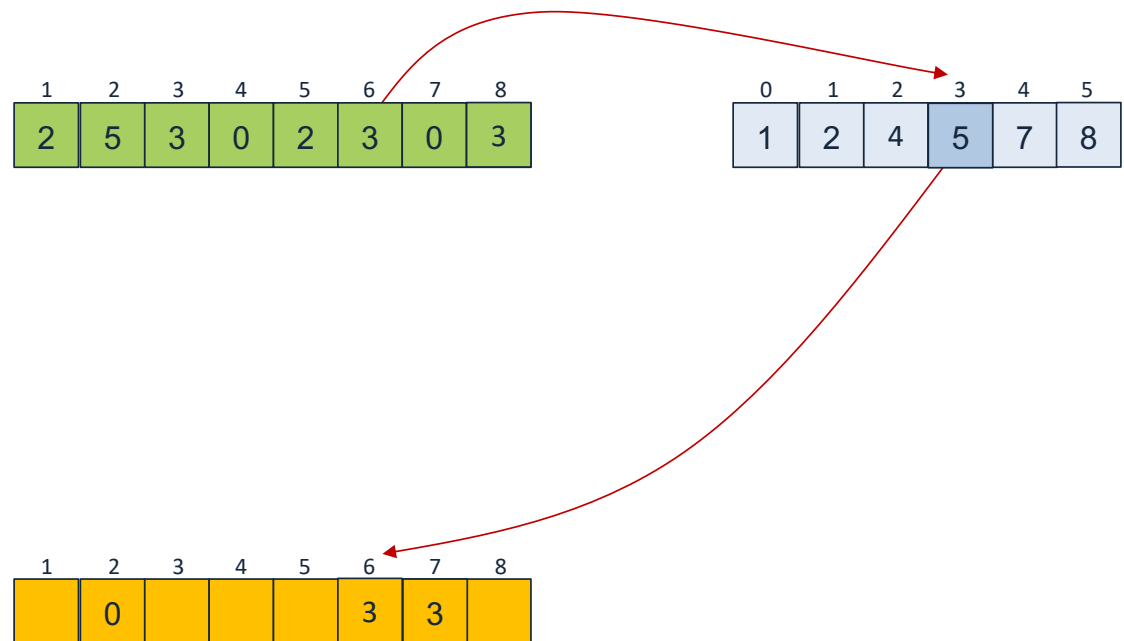
4. Schritt:

Zuletzt wird das Ergebnis-Feld B befüllt.

```

11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for

```



Sortiervverfahren

Counting-Sort

Effizienzüberlegung

Im Algorithmus finden sich nur unverschachtelte Schleifen über n (Länge des Feldes A) und k . Damit ergibt sich zunächst eine Laufzeitkomplexität von

$$O(n + k)$$

Normalerweise wird der Counting-Sort verwendet in Szenarien mit $k \in O(n)$, d.h. bei wachsendem n erhöht sich die Schranke k max. linear. Daraus ergibt sich dann für den Counting-Sort

$$O(n)$$

COUNTING-SORT(A, B, k)

```
1:  $C \leftarrow \text{new Array}[0 \dots k]$ 
2: for  $i \leftarrow 0$  to  $k$  do
3:    $C[i] \leftarrow 0$ 
4: end for
5: for  $j \leftarrow 1$  to  $\text{length}(A)$  do
6:    $C[A[j]] \leftarrow C[A[j]] + 1$ 
7: end for
8: for  $i \leftarrow 1$  to  $k$  do
9:    $C[i] \leftarrow C[i] + C[i - 1]$ 
10: end for
11: for  $j \leftarrow \text{length}(A)$  downto 1 do
12:    $B[C[A[j]]] \leftarrow A[j]$ 
13:    $C[A[j]] \leftarrow C[A[j]] - 1$ 
14: end for
```

Vielen Dank für die Aufmerksamkeit!